1. Methodology

1.1. Introduction

This chapter details the research design, architectural implementation, and evaluation protocols employed to construct the proposed tri-modal fusion model for M&A synergy prediction. The methodology is structured to operationalise the theoretical findings from Section 1, translating the need for "multimodal fusion" into a rigorous engineering specification.

The chapter proceeds as follows. Section 1.2 establishes the research philosophy and epistemological stance. Section 1.3 formalises the three hierarchical hypotheses. Section 1.4 describes all data sources and collection pipelines. Section 1.5 covers cleaning, normalisation, and temporal splitting. Section 1.6 defines the three feature blocks (Financial, Textual, Graph). Section 1.7 specifies the baseline and fusion model architectures. Section 1.8 derives the CAR target variable. Section 1.9 details the hypothesis-testing protocol, and Section 1.11 addresses limitations and ethical considerations.

1.2. Research Philosophy and Design

This study adopts a **post-positivist** epistemological stance, treating M&A synergy as a latent, probabilistic construct approximated through market reactions and structured inter-firm relationships. While acknowledging that markets are not perfectly efficient, the research operates within the semi-strong form of the Efficient Market Hypothesis (Fama, 1991), wherein publicly available information — financial fundamentals, regulatory filings, and network topology — constitutes a viable predictor signal. The overarching research design is **quantitative and deductive**: three *a priori* hypotheses (H1, H2, H3) are specified before analysis and tested through controlled ablation experiments.

The study employs a **cross-sectional observational design**. Because M&A deals are historical and non-repeatable, no experimental manipulation is possible; causal inference is instead approximated through systematic covariate control, ablation modelling, and statistical hypothesis testing, following standard practice in empirical corporate finance (MacKinlay, 1997).

An **Experimental Prototyping SDLC** governs the engineering programme. Non-deterministic model outputs and stochastic training dynamics demand reproducible seeding, versioned artefacts, and isolated ablation configurations rather than a traditional waterfall build process. Each experimental variant is fully specified in a YAML configuration (e.g., `full_fusion.yaml`) that pins hyperparameters, random seeds, feature subsets, and evaluation splits.

1.3. Research Hypotheses

Three hierarchical hypotheses structure the empirical programme. Together they form a logical escalation: H1 tests whether the graph stream improves directional discrimination (classification) above financials; H2 tests whether the text stream adds semantic signal for the same; H3 tests whether all three streams fused together outperform every individual modality in both regression (magnitude) and classification (direction).

Predicting both the sign and the magnitude of CAR serves two distinct analytical purposes. Predicting the **magnitude** (via regression) is notoriously difficult due to competing bids, information asymmetry, and macro shocks, but it is necessary to test structural variance compression (H3). Predicting the **sign** (via classification) is highly tractable and systematically links to practical deal advisory: whether the deal is a net positive or a net negative (H1, H2). The architecture therefore formulates CAR as a **dual target**: a continuous variable for

variance analysis, and a thresholded binary label (1 if CAR > 0, 0 otherwise) for directional discrimination.

ID	Name	Formal Statement
H1	Topological Alpha	Supply-chain network centrality metrics derived from Bloomberg SPLC carry statistically significant predictive signal for acquirer CAR direction, incremental to financial fundamentals alone (statistically significant AUC-ROC improvement on held-out test set, $p < 0.05$ by paired t-test across cross-validation folds).
H2	Semantic Divergence	The cosine distance between acquirer and target FinBERT embeddings of their respective 10-K MD&A sections is a significant predictor of post-acquisition CAR ($p < 0.05$, Pearson/Spearman correlation; confirmed by ablation).
H3	Topological Arbitrage	The full tri-modal fusion (Financial + Text + Graph) significantly outperforms all single-modality baselines on held-out CAR prediction. Success criterion: (a) Regressor Pipeline — statistically significant MAE and R^2 improvement, $p < 0.05$, Cohen's $d > 0.2$; (b) Classifier Pipeline — AUC-ROC improvement as secondary confirmatory evidence. H3 is confirmed if and only if criterion (a) is met.

Table 1: Research Hypotheses

1.4. Data Sources and Collection

1.4.1. M&A Deal Universe

The primary dataset is sourced from the **London Stock Exchange Group (LSEG) Refinitiv** database, which provides deal-level financial attributes for completed M&A transactions. Five raw CSV exports are merged via `scripts/data/build_combined_dataset.py` into `data/interim/ma_combined.csv`.

The deal universe is restricted to:

- Completed acquisitions of publicly listed US targets by publicly listed US acquirers.
- Transactions announced between 2000 and 2023.
- Deal values exceeding USD 50 million (sufficient market microstructure data for reliable CAR estimation).

These filters follow established practice (Betton et al., 2008) and ensure a minimum of 120 trading days in the estimation window.

1.4.2. Equity Return Data

Daily equity returns for acquirer firms and the S&P 500 benchmark are retrieved via **yfinance** (Bloomberg ticker conversion handled by `pull_car_data.py`). Returns are aligned to deal announcement dates and stored in a long-format time series (`timeseries_long.csv`) with a

`rel_day` field denoting trading-day distance from announcement (Day 0 = first trading day on or after the announcement date, forward-fill rule). Failed ticker lookups are retried with fuzzy-matching heuristics.

1.4.3. Textual Data (SEC EDGAR)

10-K annual filings for acquirer firms are retrieved from the **SEC EDGAR full-text search API**, targeting the MD&A (Item 7, `item_7_mda.txt`) and Risk Factors (Item 1A, `item_1a_risk.txt`) sections for the fiscal year immediately preceding each announcement. Extraction is handled by `src/features/edgar.py`, with download provenance logged in `data/external/edgar/download_log.csv`.

1.4.4. Supply-Chain Network Data

Inter-firm supply-chain relationships are sourced from **Bloomberg SPLC** (Supply Chain Analysis), which maps disclosed customer–supplier relationships for publicly listed firms. The SPLC data is merged with the deal universe via `scripts/data/merge_splc_data.py`, matching on Bloomberg ticker symbols. This forms the edge set for the heterogeneous graph constructed in Section 1.6.3.

1.5. Data Preprocessing

1.5.1. Cleaning and Quality Control

Raw LSEG exports undergo systematic cleaning in `scripts/data/data_cleaning.py`: date parsing and standardisation; deduplication of records sharing the same acquirer–target–announcement-date triplet; removal of records with missing acquirer ticker or announcement date; and currency normalisation to USD using period-end exchange rates.

1.5.2. Feature Engineering and Normalisation

Financial features comprise 56 ratio-level variables spanning acquirer and target leverage, liquidity, profitability, and deal structure characteristics. The preprocessing pipeline applies:

1. **Winsorisation** at the 1st and 99th percentile to bound outlier influence.
2. **Z-score standardisation** (zero mean, unit variance) computed on training-set statistics *only*, then applied to validation and test sets.
3. **Stratified temporal splitting** (70 / 15 / 15) by announcement year to prevent temporal leakage.

1.5.3. Temporal Splitting and Event-Window Embargo

Deals are sorted chronologically and partitioned into training (2000–2016), validation (2017–2019), and test (2020–2023) sets based on announcement year. This strict temporal ordering ensures the model never trains on information post-dating the validation or test periods.

Temporal Dataset Partition

Partition	Years	Split	Role
Training	2000–2016	70%	Model fitting
Validation	2017–2019	15%	Hyperparameter tuning + early stopping
Test	2020–2023	15%	Final held-out evaluation

Embargo: An 11-trading-day gap is enforced at each temporal boundary. Any deal announced within 11 days of a split boundary is excluded from both adjacent partitions, preventing CAR event-window overlap (cf. López de Prado, 2018).

Table 2: Temporal splits and embargo design

Concretely, because the event window spans $[-5, +5]$ trading days, two deals whose announcement dates differ by fewer than 11 trading days share overlapping market-return sequences in their CAR calculations. If one such deal falls in the training set and the other in validation, the model can implicitly learn return correlations that exist only because of calendar proximity — the **Overlapping Outcomes** problem formalised by López de Prado.The 11-day embargo eliminates this cross-contamination by construction.

1.5.4. Missing Data Strategy

Features with $> 40\%$ missing values are excluded. For remaining missing values, median imputation is applied to continuous features and mode imputation to categorical indicators. All imputation statistics are fitted on the training set only.

1.6. Feature Extraction

The three feature blocks are summarised in Table 3.

Block	Modality	Source	Construction (Fusion Pipeline)
A	Financial	LSEG Refinitiv	56-column ratio matrix → Winsorise → z-score → `ProjectionHead` ($\mathbb{R}^{56} \to \mathbb{R}^{64}$, linear + ReLU)
B	Textual (FinBERT)	SEC EDGAR 10-K (Item 7 + Item 1A)	FinBERT tokenisation (512-token chunks, stride = 256) → `[CLS]` from penultimate layer → mean-pool across chunks → $\mathbb{R}^{768}$ per section → PCA compression (fit on train only): $\mathbb{R}^{768} \to \mathbb{R}^{64}$ per section → concatenate MD&A + RF vectors → $\mathbb{R}^{128}$ total. *Note: pairwise cosine similarity between acquirer and target section embeddings is computed separately as a scalar predictor for the H2 OLS test; it is not an input to the fusion model.*
C	Graph (HeteroGraphSAGE)	Bloomberg SPLC	HeteroConv (2-layer SAGEConv, separate per edge type) on `(company, supplies, company)` and `(company, buys_from, company)` edges → 64-dim node embedding per firm. *Deals without SPLC coverage receive a zero vector; the graph stream is masked via `has_graph = False` in the fusion model.*

Table 3: Feature block definitions and fusion-pipeline construction steps.

1.6.1. Block A — Financial Features

The financial feature vector $\boldsymbol{h}_F \in \mathbb{R}^{d_F}$ is constructed directly from the standardised preprocessing output ($d_F = 56$). For baseline models (Ridge Regression, ElasticNet, XGBoost), $\boldsymbol{h}_F$ is used directly. For the MLP and fusion models, it passes through a `ProjectionHead` — a linear layer followed by ReLU — that maps $\boldsymbol{h}_F$ to a lower-dimensional embedding $\hat{\boldsymbol{h}}_F \in \mathbb{R}^{64}$ before concatenation.

1.6.2. Block B — Textual Features (FinBERT)

Each acquirer firm's MD&A (Item 7) and Risk Factors (Item 1A) text is processed through **FinBERT** (`ProsusAI/finbert`) (Araci, 2019), a BERT-base architecture fine-tuned on financial communications corpora. The exact pipeline, as implemented in `src/features/text.py`, is:

1. **Chunking.** The raw section text is tokenised (no truncation) and split into overlapping 512-token windows with a stride of 256 tokens, reserving positions 0 and 511 for the `[CLS]` and `[SEP]` tokens respectively.

2. **Extraction.** For each chunk, the `[CLS]` token representation is taken from the **penultimate transformer layer** (`hidden_states[-2]`), yielding a 768-dimensional vector.
3. **Pooling.** Chunk-level vectors are **mean-pooled** across all chunks to produce a single $\boldsymbol{h}_T \in \mathbb{R}^{768}$ per section.
4. **PCA compression.** Each section's embedding matrix is independently PCA-compressed:

$$\boldsymbol{h}_T^{\text{section}} \in \mathbb{R}^{768} \xrightarrow{\text{PCA, fit on train only}} \boldsymbol{p}^{\text{section}} \in \mathbb{R}^{64}$$

Separate PCA models are fitted for MD&A and Risk Factors, serialised to `data/processed/pca_models.pkl` for reproducible inference. This compression reduces the 1536-dimensional raw concatenation to 128 dimensions while retaining maximum explained variance.

1. **Concatenation.** The two 64-dimensional section vectors are concatenated into the final textual embedding:

$$\boldsymbol{h}_T = \left[\boldsymbol{p}^{\text{MDA}} \parallel \boldsymbol{p}^{\text{RF}}\right] \in \mathbb{R}^{128}$$

FinBERT's ≈ 110M parameters are **frozen** throughout all downstream training to prevent overfitting given the limited M&A sample size; only the downstream projection heads are trained.

1.6.2.1. Cosine Similarity (H2 Test Only)

For the H2 semantic-divergence hypothesis, a pairwise cosine similarity score is computed **separately** between the acquirer and target's section embeddings, *after* PCA compression:

$$\text{SemanticDiv}_i = 1 - \frac{p_{\text{acq}}^{\text{MDA}} \cdot p_{\text{tgt}}^{\text{MDA}}}{\|p_{\text{acq}}^{\text{MDA}}\| \cdot \|p_{\text{tgt}}^{\text{MDA}}\|}$$

This scalar divergence score is used **exclusively** as the independent variable in the H2 OLS regression. It is **not** an input to the fusion model. The distinction is critical: cosine distance is a *relationship-level* scalar characterising strategic fit between two firms, while the fusion model requires *firm-level* vectors to learn independent acquirer representations.

1.6.3. Block C — Graph Features (HeteroGraphSAGE)

The inter-firm supply-chain network is constructed as a heterogeneous graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{T}_v, \mathcal{T}_e)$ from Bloomberg SPLC data, using PyTorch Geometric's `HeteroData` object (`scripts/graphs/build_hetero_graph.py`).

1.6.3.1. Graph Structure

- **Node type.** A single node type `company` represents each publicly listed firm present in the SPLC dataset. Node features are initialised with degree centrality, betweenness centrality, and the standardised financial feature vector.
- **Edge types.** Two directed relationship types are encoded:

Edge Type	Direction	Semantics
`supplies`	supplier → customer	Firm A discloses Firm B as a customer in SPLC; directional dependency.
`buys_from`	customer → supplier	Inverse of `supplies` encodes upstream procurement risk.

All edges represent **currently active, disclosed supply-chain relationships** sourced from SPLC. No M&A-derived edges (e.g., historical acquisition links) are included in the adjacency matrix. This design choice eliminates any possibility of structural target leakage: the model has no graph path connecting an acquirer to its deal target, because no such edge class exists.

> **Leakage Note.** The absence of acquisition edges is intentional and architecturally guarantees zero structural target leakage. Supply-chain relationships exist independently of M&A outcomes and persist whether or not a deal completes. The model cannot "see" the deal being predicted through the graph.
>
> Four edge types were considered in the project design phase (`supplier_of`, `customer_of`, `competitor_of`, `acquires`). The implemented scope uses two SPLC-sourced types. Competitor and historical acquisition edges are a natural extension discussed in Section 1.11.

1.6.3.2. HeteroGraphSAGE Model

A 2-layer **Heterogeneous GraphSAGE** model is trained via self-supervised link prediction on the supply-chain graph, as implemented in `scripts/graphs/train_hetero_graph.py`:

HeteroGraphSAGE Architecture

Layer	Operation	Detail
Conv 1	`HeteroConv`	Separate `SAGEConv(in → 128)` per edge type; mean aggregation across types
Activation	ReLU + Dropout	$p = 0.3$; applied per-type after Layer 1
Conv 2	`HeteroConv`	Separate `SAGEConv(128 → 64)` per edge type; mean aggregation
Output	Node embedding	$\boldsymbol{h}_G \in \mathbb{R}^{64}$ per company node

Training: self-supervised link prediction (binary cross-entropy), negative sampling per edge type, Adam lr = 0.01, 200 epochs. Final embeddings extracted via `model.encode()` with full edge set.

Table 4: HeteroGraphSAGE architecture and training configuration.

The key architectural innovation over a homogeneous GraphSAGE is that `supplies` and `buys_from` edges learn **independent SAGEConv weight matrices**, enabling the model to distinguish upstream procurement signals from downstream customer dependency signals during message passing. Node embeddings are extracted and stored in `data/interim/hetero_graph_embeddings.csv` (64-dimensional, one row per company ticker), then merged into the training dataset via deal–ticker matching.

1.7. Model Architecture

1.7.1. Baseline Models

Four baselines are trained on Block A features only:

Model	Variant ID	Purpose
Naïve Mean	M0	Lower bound; predicts training-set mean CAR for all deals.
Ridge Regression	M1	Linear baseline; controlled regularisation.
ElasticNet	M2	Feature selection; identifies redundant financial ratios.
XGBoost	M3	Non-linear financial-only ceiling; tests H1/H2/H3 incremental gain.
Financial + Text	M4	Ablation: removes graph stream; tests H2 in isolation.
Financial + Graph	M5	Ablation: removes text stream; tests H1 in isolation.
Full Fusion (F+T+G)	M6	Primary model; tests H3.

Table 5: Model variants for ablation experiments.

1.7.2. Fusion Model Architecture

The primary model is the **late-fusion tri-modal architecture** implemented in `src/models/fusion.py`. Each active stream passes through its own `ProjectionHead` (linear + ReLU), and the resulting embeddings are concatenated:

$$\boldsymbol{z}_i = [\boldsymbol{h}_F \parallel \boldsymbol{h}_T \parallel \boldsymbol{h}_G] \in \mathbb{R}^{d_{F'}+d_{T'}+d_{G'}}$$

where $d_{F'} = 64$, $d_{T'} = 64$, $d_{G'} = 32$ by default. To rigorously evaluate both the magnitude of synergy and the practical investment decision, the architecture implements a **Dual-Evaluation Framework**. The concatenated feature vector $\boldsymbol{z}_i$ serves as the input to two parallel, independent training pipelines:

- **Regressor Pipeline (H3):** Includes a two-layer MLP with a **linear** output activation (and XGBRegressor), optimised via Mean Squared Error (MSE) to predict continuous CAR. This tests structural variance explanations.
- **Classifier Pipeline (H1, H2):** Includes a two-layer MLP with a **sigmoid** output activation (and XGBClassifier), optimised via Binary Cross-Entropy (BCE) to predict the binary direction of CAR (1 if CAR > 0, 0 otherwise). This tests practical deal advisory discrimination.

Both pipelines share the same upstream concatenated representation $\boldsymbol{z}_i$, meaning the feature streams are extracted once and evaluated across both objectives.

Tri-Modal Fusion Architecture

Stream A	**Stream B**	**Stream C**
Financial	Textual	Graph
$\boldsymbol{h}_F \in \mathbb{R}^{56}$	$\boldsymbol{h}_T \in \mathbb{R}^{128}$	$\boldsymbol{h}_G \in \mathbb{R}^{64}$
$\downarrow$ ProjectionHead	$\downarrow$ ProjectionHead	$\downarrow$ ProjectionHead
$\hat{\boldsymbol{h}}_F \in \mathbb{R}^{64}$	$\hat{\boldsymbol{h}}_T \in \mathbb{R}^{64}$	$\hat{\boldsymbol{h}}_G \in \mathbb{R}^{32}$

$\downarrow$ Concatenate $\rightarrow \boldsymbol{z}_i \in \mathbb{R}^{160}$

Regressor Pipeline (H3)	**Classifier Pipeline (H1, H2)**
`train_models.py`	`train_classifier.py`
MLP + XGBRegressor + Linear activation	MLP + XGBClassifier + Sigmoid activation
$\hat{y}_i \in \mathbb{R}$ (CAR magnitude)	$\hat{p}_i \in [0, 1]$ (CAR direction)
Loss: MSE	Loss: BCE

Streams with missing data (`has_graph=False`) contribute a zero vector. The modular design enables controlled ablation by disabling any stream subset.

Listing 1: Tri-modal late-fusion architecture (src/models/fusion.py).

1.7.3. Training Configuration

All PyTorch models are trained with:

- **Optimiser:** AdamW with cosine annealing learning-rate schedule with warm restarts.
- **Loss:** Two objectives trained in parallel: MSE for the Regressor Pipeline (continuous CAR); Binary Cross-Entropy for the Classifier Pipeline (binary CAR direction). Huber loss sensitivity analysis is reported alongside MSE.
- **Early stopping:** Validation MAE with patience of 15 epochs.
- **Batch size:** 64.
- **Reproducibility:** Fixed random seed via `set_seed()` in `src/training/trainer.py`.
- **Device:** CUDA / Apple MPS / CPU auto-selected via `src/config.py`.

1.8. Target Variable: Cumulative Abnormal Return

The target variable y_i for each deal is the **Cumulative Abnormal Return (CAR)** over the symmetric event window $[-5, +5]$ trading days relative to announcement date, computed via the standard market model (Brown & Warner, 1985; MacKinlay, 1997). This section describes the full two-stage pipeline: Stage 1 derives **actual** CAR values from raw market data using OLS; Stage 2 trains the fusion model to **predict** CAR from pre-announcement features and evaluates predicted CAR against actual CAR on held-out deals.

Two-Stage CAR Pipeline

Stage 1 — OLS Event Study

(`compute_car.py`)

1. Download acquirer + S&P 500 daily prices (yfinance)
2. Compute log returns: $R_t = \ln(P_t/P_{t-1})$
3. Estimation window $[-200, -20]$: fit OLS
   $R_{it} = \hat{\alpha}_i + \hat{\beta}_i R_{mt}$
4. Event window $[-5, +5]$: compute AR
   $AR_{it} = R_{it} - \left(\hat{\alpha}_i + \hat{\beta}_i R_{mt}\right)$
5. Sum residuals:
   $\mathrm{CAR}_i = \sum_{t=-5}^{+5} AR_{it}$
6. Merge into `final_car_dataset.csv`

`car_m5_`

Stage 2 — Supervised Prediction

(`training_utils.py`)

Input features (pre-announcement only):
$\boldsymbol{z}_i = [\boldsymbol{h}_F \parallel \boldsymbol{h}_T \parallel \boldsymbol{h}_G]$

Fusion model predicts:
$\hat{y}_i = f_\theta(\boldsymbol{z}_i)$

Evaluated against actual CAR:
$y_i = \mathrm{CAR}_i =$ `car_m5_p5` column

Losses (training):\
$\mathcal{L}_{\mathrm{MSE}} = \frac{1}{N}\sum_i (y_i - \hat{y}_i)^2 + \lambda \|\theta\|_2^2$ (Regressor)\
$\mathcal{L}_{\mathrm{BCE}} = -\frac{1}{N}\sum_i \Big[y_{\{\mathrm{bin},i\}} \log \hat{p}_i + \big(1 - y_{\{\mathrm{bin},i\}}\big) \log(1 - \hat{p}_i)\Big] + \lambda \|\theta\|_2^2$ (Classifier)\
\
Held-out metrics:\
MAE, RMSE, R^2 (regression); AUC-ROC, F1 (classification)

Stage 1 outputs are fixed market-derived labels independent of any model. Stage 2 uses pre-announcement features only — no post-deal information enters $\boldsymbol{z}_i$.

Listing 2: The two-stage CAR pipeline: Stage 1 computes actual CAR via OLS event study; Stage 2 trains and evaluates the fusion model against those labels.

1.8.1. Stage 1: OLS Market Model

The market model is estimated over the **estimation window** $[-200, -20]$ trading days (180-day window, minimum 120 valid observations) using OLS as implemented in `scripts/data/compute_car.py` via `scipy.stats.linregress`:

$$R_{it} = \alpha_i + \beta_i R_{mt} + \varepsilon_{it}$$

where:

- $R_{it} = \ln\big(P_{it}/P_{i,t-1}\big)$ is the acquirer log return on trading day t,
- R_{mt} is the S&P 500 (SPX) log return on the same day,
- $\hat{\alpha}_i$ is the estimated intercept (abnormal return in absence of market movement),
- $\hat{\beta}_i$ is the estimated systematic risk loading (market beta).

The OLS estimators are:

$$\hat{\beta}_i = \frac{\mathrm{Cov}(R_i, R_m)}{\mathrm{Var}(R_m)}$$

$$\hat{\alpha}_i = \overline{R}_i - \hat{\beta}_i \overline{R}_m$$

A gap window $[-19, -6]$ between estimation and event windows is excluded from both calculations, preventing estimation-period price dynamics from contaminating the event-window benchmark.

Event Study Timeline (Trading Days)

Estimation Window	**Gap (excl.)**	**Day 0**	$\longleftarrow$	**Event Window**
$[-200, -20]$ OLS: fit $\hat{\alpha}_i, \hat{\beta}_i$	$[-19, -6]$ excluded	**Announcement** (Day 0)		$[-5, +5]$ sum $AR_{it} \rightarrow \mathrm{CAR}_i$

Table 6: Event study timeline. The gap prevents estimation-period contamination of the CAR window.

1.8.2. Stage 1: Abnormal Returns and CAR

With $\hat{\alpha}_i$ and $\hat{\beta}_i$ estimated on the estimation window, **abnormal returns** in the event window $\mathcal{T} = \{-5, ..., +5\}$ are the residuals between actual and model-predicted returns:

$$AR_{it} = R_{it} - \left(\hat{\alpha}_i + \hat{\beta}_i R_{mt}\right)$$

AR_{it} represents the return attributable to deal-specific information (announcement effect) after stripping out normal market co-movement. **CAR is the cumulative sum of these residuals** over the full event window:

$$\mathrm{CAR}_i = \sum_{t=-5}^{+5} AR_{it}$$

This produces a single scalar per deal stored as column `car_m5_p5` in `data/processed/final_car_dataset.csv`. A positive CAR indicates the market rewarded the acquisition announcement; a negative CAR indicates value destruction.

1.8.3. Stage 2: Model Prediction vs. Actual CAR

The column `car_m5_p5` (set as `TARGET_COL` in `scripts/training/training_utils.py`) is the sole prediction target for all model variants. The fusion architecture minimises two distinct objective functions depending on the evaluation pipeline:

1. Continuous Target (Regression Pipeline — H3): For variance analysis, the Regressor Pipeline minimises Mean Squared Error against the raw continuous CAR ($y_i = \mathrm{CAR}_i \in \mathbb{R}$):

$$\mathcal{L}_{\mathrm{MSE}(\theta)} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2 + \lambda \, \|\theta\|_2^2$$

2. Binary Target (Classification Pipeline — H1, H2): For directional discrimination, the Classifier Pipeline minimises Binary Cross-Entropy against the thresholded label $y_{\{\mathrm{bin},i\}} = \mathbb{1}[\mathrm{CAR}_i > 0] \in \{0, 1\}$:

$$\mathcal{L}_{\mathrm{BCE}}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \left[y_{\{\mathrm{bin},i\}} \log \hat{p}_i + \left(1 - y_{\{\mathrm{bin},i\}}\right) \log(1 - \hat{p}_i) \right] + \lambda \, \|\theta\|_2^2$$

where $\hat{p}_i = \sigma\left(f_{\theta(\boldsymbol{z}_i)}\right)$ is the sigmoid-activated synergy probability. All evaluation metrics are then computed by comparing predictions against y_i (regression) or $y_{\{\mathrm{bin},i\}}$ (classification) on the held-out test set:

Metric	Formula	Interpretation
MAE	$\frac{1}{N}\sum_i \lvert y_i - \hat{y}_i \rvert$	Primary metric; interpretable in percentage-point CAR terms.
RMSE	$\sqrt{\frac{1}{N}\sum_i (y_i - \hat{y}_i)^2}$	Penalises large mispredictions more than MAE.
R^2	$1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \overline{y})^2}$	Proportion of CAR variance explained by the model.
Huber	$\sum_i \mathcal{H}_\delta(y_i - \hat{y}_i)$	Robust to outlier returns; sensitivity analysis only.
Dir. Accuracy	$\frac{1}{N}\sum_i \mathbb{1}[\text{sign}(\hat{y}_i) = \text{sign}(y_i)]$	**Classification Pipeline.** Did the model predict the deal direction correctly?
AUC-ROC	$\int_0^1 \text{TPR}(\text{FPR})\, d(\text{FPR})$	**Primary classification metric.** Threshold-invariant measure of the model's ability to rank value-creating deals (CAR > 0) above value-destroying ones.
F1-Score	$2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$	Harmonic mean of precision and recall. Evaluates robustness under class imbalance in the CAR-positive vs. CAR-negative split.

Table 7: Evaluation metrics. Regression metrics (MAE, RMSE, R^2, Huber) apply to the Regressor Pipeline; classification metrics (Dir. Accuracy, AUC-ROC, F1) apply to the Classifier Pipeline.

The critical design guarantee is that **no feature in $\boldsymbol{z_i}$ is derived from post-announcement data**. Financial ratios $\boldsymbol{h}_F$ use the most recent pre-announcement fiscal year; FinBERT embeddings $\boldsymbol{h}_T$ use the 10-K filed before announcement; graph embeddings $\boldsymbol{h}_G$ use SPLC relationships that are structurally independent of deal outcomes. The model therefore predicts market reaction from purely pre-deal information — the economically meaningful and leakage-free formulation.

1.9. Hypothesis Testing

Each hypothesis is tested through model ablation combined with statistical significance testing:

- **H1 (Topological Alpha):** Compare M5 (`financial_graph.yaml`) vs. M3 (`financial_only.yaml`) on the **Classifier Pipeline**. A paired t-test across cross-validation folds on held-out AUC-ROC scores assesses whether the graph stream yields a statistically significant improvement in directional discrimination ($p < 0.05$).

- **H2 (Semantic Divergence):** H2 is confirmed if and only if **both** conditions hold: (a) a statistically significant Pearson/Spearman correlation between `SemanticDiv_i` and CAR_i ($p < 0.05$, primary test); and (b) M4 (`financial_text.yaml`) yields a statistically significant AUC-ROC improvement over M3 (`financial_only.yaml`) by paired t-test across cross-validation folds (confirmatory ablation). Neither condition alone is sufficient.
- **H3 (Topological Arbitrage):** M6 (`full_fusion.yaml`) is compared against all single-modality baselines. Effect size (Cohen's d on hold-out error distributions) and R^2 improvement are reported alongside p-values.

All tests use a significance threshold of $\alpha = 0.05$ with **Bonferroni correction** applied across the three hypothesis tests to control the family-wise error rate ($\alpha_{\text{corrected}} = 0.0167$).

1.10. Evaluation Metrics

Evaluation metrics are partitioned by prediction head, as formalised in Table 7. The **Regressor Pipeline** (H3) is evaluated on continuous CAR predictions using MAE (primary), RMSE, R^2, and Huber loss (sensitivity). MAE is the primary regression metric given its interpretability in percentage-point CAR terms and its robustness to the outlier return distribution.

The **Classifier Pipeline** (H1, H2) is evaluated on binary CAR-direction predictions using AUC-ROC (primary), F1-Score, and Directional Accuracy. AUC-ROC is the primary classification metric because it is threshold-invariant and directly measures the model's ability to rank value-creating deals above value-destroying ones — the economically actionable output for deal advisory. F1-Score is reported as a secondary metric to assess robustness under class imbalance in the CAR-positive vs. CAR-negative split.

1.11. Ethical Considerations and Limitations

All data used is commercially licensed (LSEG, Bloomberg) and contains no personally identifiable information. The study does not involve human participants.

Key methodological limitations include:

1. **SPLC Disclosure Bias.** The supply-chain network captures only disclosed relationships, potentially biasing graph features toward larger firms with more extensive reporting obligations. Smaller firms may have sparser neighbourhoods that understate their true network centrality.
2. **Frozen FinBERT Embeddings.** Frozen weights may not fully capture M&A-specific language not present in FinBERT's training corpus. Fine-tuning on a domain-specific financial corpus represents a natural extension.
3. **Market-Model Beta Stationarity.** The OLS market model assumes beta is stationary over the estimation window, which may be violated for firms undergoing strategic repositioning pre-deal.
4. **US-Listed Sample.** The sample is restricted to US-listed firms, limiting generalisability to cross-border or private-equity transactions.
5. **Edge Type Scope.** The implemented graph uses two SPLC-derived edge types (`supplies`, `buys_from`). Historical acquisition edges and competitor-of edges were considered in the design but not implemented within the project's data budget. Their inclusion, using survivorship-bias-corrected datasets, is a natural direction for future work.

Bibliography

Araci, D. (2019,). *FinBERT: Financial Sentiment Analysis with Pre-trained Language Models*. http://arxiv.org/abs/1908.10063

Betton, S., Eckbo, B. E., & Thorburn, K. S. (2008). Corporate Takeovers. In *Handbook of Empirical Corporate Finance: Handbook of Empirical Corporate Finance*. Elsevier. https://doi.org/10.1016/B978-0-444-53265-7.50007-X

Brown, S. J., & Warner, J. B. (1985). Using Daily Stock Returns: The Case of Event Studies. *Journal of Financial Economics*, *14*(1), 3–31. https://doi.org/10.1016/0304-405X(85)90042-X

Fama, E. F. (1991). Efficient Capital Markets: II. *The Journal of Finance*, *46*(5), 1575–1617. https://doi.org/10.2307/2328565

MacKinlay, A. C. (1997). Event Studies in Economics and Finance. *Journal of Economic Literature*, *35*(1), 13–39.